

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 20%

QUESTIONS:15 Total Marks:25 Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Prudhvi Raj.B(192324274) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:prudhvi raj.b

Annexure A

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.
2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.
3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.
4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.
5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Pseudocode Writing Task – Algorithm Design

1. Dataset Preprocessing Algorithm

Input: Raw dataset file `input.csv`

Output: Cleaned dataset `cleaned.csv`

ALGORITHM PreprocessDataset

INPUT: Dataset file

OUTPUT: Cleaned dataset

BEGIN

READ input.csv INTO DataFrame DF

IF file cannot be opened THEN

 DISPLAY "File Error"

 STOP

END IF

DISPLAY dataset information

// Handle missing values

FOR each numerical column IN DF DO

 REPLACE missing values WITH column median

END FOR

FOR each categorical column IN DF DO

 REPLACE missing values WITH column mode

END FOR

// Remove duplicate records

REMOVE duplicate rows FROM DF

// Standardize categorical values

FOR each categorical column IN DF DO

 REMOVE leading and trailing spaces

 CONVERT values TO lowercase

 STANDARDIZE equivalent categories

END FOR

// Filter invalid numerical entries

FOR each numerical column IN DF DO

 IDENTIFY values outside valid range

 REMOVE or replace invalid values

END FOR

// Normalize numerical columns

FOR each numerical column IN DF DO

 MIN ← minimum value

 MAX ← maximum value

 IF MAX ≠ MIN THEN

$DF[column] \leftarrow (DF[column] - MIN) / (MAX - MIN)$

 END IF

END FOR

// Indexing and final filtering

RESET DataFrame index

REMOVE rows that still contain invalid or missing values

IF DF is empty THEN

```
    DISPLAY "No valid records available"
    STOP
END IF

SAVE DF AS cleaned.csv

DISPLAY "Dataset preprocessing completed"

END
```

Efficiency: Vectorized DataFrame operations, column-wise processing, and avoiding unnecessary copies improve computational efficiency.

2. Statistical Measures Algorithm

Input: Large numerical dataset

Output: Mean, median, standard deviation, and variance

ALGORITHM CalculateStatistics

INPUT: Numerical dataset

OUTPUT: Mean, Median, Standard Deviation, Variance

BEGIN

```
    READ dataset INTO array A
```

```
    IF dataset cannot be read THEN
        DISPLAY "Dataset Error"
        STOP
    END IF
```

```
    REMOVE or ignore missing values FROM A
```

```
    IF length(A) = 0 THEN
        DISPLAY "Empty dataset"
        STOP
    END IF
```

```
    // Filter unwanted values
    REMOVE non-numerical values FROM A
```

```
    // Detect extreme outliers
    CALCULATE Q1 and Q3
    IQR ← Q3 - Q1
```

```
    LOWER_LIMIT ← Q1 - 1.5 × IQR
    UPPER_LIMIT ← Q3 + 1.5 × IQR
```

IDENTIFY values outside the limits

```
IF outlier removal is required THEN
  FILTER A using LOWER_LIMIT and UPPER_LIMIT
END IF
```

```
IF length(A) = 0 THEN
  DISPLAY "No valid values"
  STOP
END IF
```

```
// Calculate aggregation values
SUM ← SUM(A)
COUNT ← length(A)
```

```
MEAN ← SUM / COUNT
```

```
SORT A
```

```
IF COUNT is odd THEN
  MEDIAN ← middle element of A
ELSE
  MEDIAN ← average of two middle elements
END IF
```

```
VARIANCE ← SUM((A - MEAN)2) / COUNT
```

```
STANDARD_DEVIATION ← square root of VARIANCE
```

```
DISPLAY MEAN
DISPLAY MEDIAN
DISPLAY VARIANCE
DISPLAY STANDARD_DEVIATION
```

END

Efficiency: Array-based operations and vectorized aggregation reduce loops and improve performance for large datasets.

Note: If sample variance is required, divide by $COUNT - 1$ instead of $COUNT$.

3. Combining Multiple DataFrames

Input: Two or more related DataFrames

Output: Validated merged DataFrame

ALGORITHM CombineDataFrames

INPUT: DataFrames DF1, DF2, ..., DF_n
OUTPUT: Merged DataFrame MERGED

BEGIN

FOR each DataFrame DF_i DO

CHECK column names
CHECK data types
CHECK missing values
CHECK duplicate records

STANDARDIZE common key format

END FOR

// Align indices
RESET index of each DataFrame

// Check common key
IDENTIFY common key K

IF common key does not exist THEN
 DISPLAY "Common key not found"
 STOP
END IF

// Remove duplicate records
FOR each DataFrame DF_i DO
 REMOVE duplicate records based on K
 IF required, retain the latest valid record
END FOR

// Merge DataFrames
MERGED ← MERGE DF1 and DF2 USING key K

FOR each remaining DataFrame DF_i DO
 MERGED ← MERGE MERGED with DF_i USING key K
END FOR

// Validate integrity
CHECK duplicate keys in MERGED

IF duplicate keys exist THEN
 RESOLVE duplicate records
END IF

CHECK missing key values

IF missing key values exist THEN

```
    REMOVE or correct invalid records  
END IF
```

```
// Grouping  
GROUP MERGED BY key K  
CALCULATE required aggregates
```

```
// Sorting  
SORT MERGED BY key K
```

```
RESET index of MERGED
```

```
VERIFY number of valid records
```

```
DISPLAY "Merge completed successfully"
```

```
RETURN MERGED
```

```
END
```

Validation: Compare record counts before and after merging and check for duplicate or missing keys to ensure consistency and completeness.

4. Filtering and Sorting Algorithm

Input: Dataset, filtering conditions

Output: Filtered and sorted dataset

ALGORITHM FilterAndSortDataset

INPUT: Dataset DF

OUTPUT: Filtered and sorted DataFrame RESULT

```
BEGIN
```

```
    READ dataset INTO DataFrame DF
```

```
    IF DF cannot be loaded THEN
```

```
        DISPLAY "File Error"
```

```
        STOP
```

```
    END IF
```

```
// Validate required columns
```

```
CHECK required columns exist
```

```
IF required columns are missing THEN
```

```
    DISPLAY "Invalid Dataset"
```

```
    STOP
```



```

END IF

// Create filtering conditions
CONDITION1 ← value >= minimum_value
CONDITION2 ← value <= maximum_value
CONDITION3 ← category IN required_categories

// Combine conditions
FINAL_CONDITION ← CONDITION1 AND
                  CONDITION2 AND
                  CONDITION3

// Apply Boolean indexing
RESULT ← DF[FINAL_CONDITION]

IF RESULT is empty THEN
    DISPLAY "No records satisfy conditions"
    STOP
END IF

// Sort filtered results
SORT RESULT BY required_column
SET sorting order as ascending or descending

RESET RESULT index

DISPLAY RESULT

SAVE RESULT if required

END

```

Large Dataset Optimization

```

IF dataset is very large THEN
    READ only required columns
    APPLY filters as early as possible
    PROCESS data in chunks
    AVOID unnecessary copies
    SORT only the filtered records
END IF

```

Conclusion: Boolean indexing efficiently applies multiple conditions, while sorting the reduced dataset minimizes computation and memory usage.

5. Complete Data Analysis Workflow

Input: One or more data files

Output: Cleaned data, summary table, and plots

ALGORITHM CompleteDataAnalysis

INPUT: Data files

OUTPUT: Cleaned DataFrame, Summary DataFrame, Plots

BEGIN

// Stage 1: File Input

GET list of input files

FOR each file DO

TRY

 READ file INTO temporary DataFrame

IF file error occurs THEN

 DISPLAY "Error reading file"

 CONTINUE to next file

END IF

CHECK file structure and columns

APPEND temporary DataFrame to main DataFrame

END FOR

IF no valid files were loaded THEN

 DISPLAY "No data available"

 STOP

END IF

// Stage 2: Data Transformation

CONVERT data INTO appropriate DataFrame types

CONVERT date columns INTO date format

CONVERT numerical columns INTO numeric format

HANDLE missing values

REMOVE duplicate records

STANDARDIZE categorical values

// Stage 3: Filtering

APPLY required conditions

FILTER invalid or unwanted records

// Stage 4: Grouping and Aggregation

GROUP DataFrame BY required category

CALCULATE:

COUNT

SUM

MEAN

MINIMUM

MAXIMUM

STORE results IN Summary DataFrame

// Stage 5: Sorting

SORT Summary DataFrame BY important measure

RESET index

// Stage 6: Visualization

CREATE appropriate plot using Summary DataFrame

ADD title

ADD X-axis label

ADD Y-axis label

DISPLAY plot

// Stage 7: Output

SAVE cleaned DataFrame

SAVE Summary DataFrame

SAVE generated plot

DISPLAY "Data analysis completed successfully"

END

Workflow

Input Files



File Validation



DataFrame Creation



Cleaning & Transformation



Filtering



Grouping & Aggregation



Sorting



Summary Data



Visualization



Final Output

Result: The algorithm produces cleaned data, aggregated summary results, and meaningful visualizations while handling file errors and inconsistent data.

Conclusion: A structured workflow combining file handling, Pandas DataFrames, filtering, grouping, aggregation, sorting, and visualization provides an efficient and reliable data analysis process.